

exp-fips203-mlkem-pke-alg13-16171820-2s1e-k4
FIPS 203 Alg.13 行 16–20: 2s1e MIX + UB 融合
(FIPS 合规 s/e 采样版)

ascendc 工程 (预研 customspec)

2026-06-19

摘要

本文档为 `examples/incubating/exp-fips203-mlkem-pke-alg13-16171820-2s1e-k4/` 的实现规格说明书 (customspec)。

算子语义: ML-KEM-1024 ($k=4$, $n=256$, $q=3329$) K-PKE.KeyGen (FIPS 203 **Algorithm 13**) 行 16–20: $\text{NTT}(\mathbf{s}), \text{NTT}(\mathbf{e}) \rightarrow \hat{t} \leftarrow \hat{A} \circ \hat{\mathbf{s}} + \hat{\mathbf{e}} \rightarrow \text{ByteEncode}_{12}(\hat{t}/\hat{\mathbf{s}})$ 。

工程壳以已调通探针 `ascendc-tests/pass-fix-f203-2s1e-alg13-16171820-vec-k4-v2/` 为唯一 AscendC 实现参照 (平面 `mat_c`、单 TPipe UB 融合、`stageHatDotOnly j→p`)。

与探针的关键差异(强制): 本 exp 的 `Host gen_data` 不得复用探针 `shortcuts(FIXED_POLY×4, (e+p) mod q` 人工变体)。 $\mathbf{s}=(s_0, \dots, s_{k-1})$ 与 $\mathbf{e}=(e_0, \dots, e_{k-1})$ 须按 FIPS 203 **Algorithm 13** 行 7–8 + **Algorithm 8** `SamplePolyCBDη` (ML-KEM-1024: $\eta_1=\eta_2=2$) 独立模拟生成, 共 8 条互不相同 (高概率) 且分布合规的时域多项式。

计划 AI Core: `KERNEL_TYPE_MIX_AIC_1_2`; `blockDim=1 (--aicore 1)`; 首版不测多核。

阶段: customspec 闭环; 禁止在本目录写可执行实现, 直至用户确认规格并明示「可以写代码」。

目录

1 元数据	2
2 定位与边界	2
2.1 用例性质	2
2.2 与探针 <code>vec-k4-v2</code> 的差异	2
2.3 本阶段不做	2
3 FIPS 203 输入采样 (Host, 强制)	3
3.1 参数 (ML-KEM-1024)	3
3.2 语义 (Algorithm 13 片段 + Algorithm 8)	3
3.3 <code>src.bin</code> 布局 (与 2s1e 设备契约一致)	3
3.4 参考实现与交叉验证 (<code>gen_data</code> 登记)	3

目录	2
4 FIPS 203 Algorithm 13 行 16–20 设备语义	4
4.1 行 16–17 (NTT)	4
4.2 行 18 (NTT 域内积 + 噪声)	4
4.3 行 19–20 (ByteEncode ₁₂)	4
5 计算语义：三段式 NTT + 平面 mat_c	5
5.1 流水线	5
5.2 平面行号（禁止混用）	5
6 数据搬运与 UB 不变量	6
6.1 驻留 vs 落盘	6
6.2 行 18 数据面	6
7 MIX 调度与 mixPass	6
7.1 FSM (blockDim=1)	6
7.2 mixPass 调试	6
8 AI Core 规模（强制专节）	6
9 I/O 与 Golden 契约	7
9.1 文件 (gen_data 与 main 逐字一致)	7
9.2 Golden 生成链 (NTT 与 v2 同构)	7
9.3 Golden 与设备 mod 解耦	7
9.4 验收	7
10 AscendC 实现规划	8
10.1 参照 fork 顺序	8
10.2 计划目录（实现阶段）	8
10.3 分步验收	8
11 全量 API 清单（主路径，CANN 9.0.0）	8
11.1 评估过但未采用	9
12 逐步覆盖率评估	9
13 附录：SIM tick 参考（910B4，非门禁）	10
14 已废弃路径	10
15 关联文档	10

1 元数据

项	内容
用例路径	examples/incubating/exp-fips203-mlkem-pke-alg13-16171820-2s1e-k4/
规格书	本文件 *-customspec.tex / .pdf
实现参照	ascendc-tests/pass-fix-f203-2s1e-alg13-16171820-vec-k4-v2/ (IMPLEMENTATION_REFERENCE.md)
原理文档	docs/notes/F203-2s1e-NTT内积UB融合技术总结.md
ByteEncode 原理	docs/notes/F203-ByteEncode12-prefetch技术总结.md
探针 (encode-only)	ascendc-tests/pass-fix-f203-2s1e-byteencode12-vec-k4/
标准	library/documents/NIST.FIPS.203.pdf Alg.13、Alg.8 (CBD)
平台	CANN 9.0.0; Atlas A2 (Ascend910B4)
核规模	blockDim=1; 1×AIC + 2×AIV / AI Core
日期	2026-06-19

2 定位与边界

2.1 用例性质

- 预研 exp-*: 验收 仅 I/O 对拍 (max_abs_diff=0)。
- 算法片段: Alg.13 行 16-20 (设备侧 MIX 核); 不含行 1-15 的 $\hat{A}$ 扩展矩阵乘、SHA3 G/H、完整 ML-KEM.KeyGen (Alg.16)。
- 采样在 Host (强制): s/e 由 scripts/gen_data.py (Python) 按 FIPS 203 生成并写入 input/src.bin; 设备只读 GM src, 不在 AscendC 上实时生成 CBD/PRF。
- 常量输入: $\hat{A}$ 仍为 Host 提供的 a_hat [16,256] (与探针相同: 固定 SEED_AHAT 文档化); 后续可换为 Alg.13 行 9-15 全链路。

2.2 与探针 vec-k4-v2 的差异

2.3 本阶段不做

- 设备实时生成 s/e (AscendC PRF/CBD、双 launch 预采样) ——归属 ascendc-tests/ 探针链, 非本 exp; Host gen_data 用 Python (可对照 liboqs)。
- Encaps/Decaps、ML-KEM.KeyGen (Alg.16)、FIPS 204。
- 多 blockDim 性能调优 (须另开 spec 修订)。
- 竖堆 mat_c、Matmul<> 融合、SHAT_PEER、frozen v1 half-basemul 路线。

项	探针 v2 (研究)	本 exp (交付向)
s 行 0..3	同一 FIXED_POLY 复制 $\times 4$	$s_0, \dots, s_3$ 各自 SamplePolyCBD ₂
e 行 4..7	基 e 上 $(e+p) \bmod q$	$e_0, \dots, e_3$ 各自 SamplePolyCBD ₂
分布	便于 NTT 手算对拍	FIPS 203 合规 CBD
行 18 $\hat{e}[p]$	每 p 不同 (人工)	每 p 不同 (标准采样)
AscendC 壳	同左栏	照抄 v2 (平面、UB 融合)

3 FIPS 203 输入采样 (Host, 强制)

3.1 参数 (ML-KEM-1024)

- 模块秩 $k=4$; $n=256$; $q=3329$ 。
- 噪声参数 $\eta_1=\eta_2=2$ (FIPS 203 表 2)。

3.2 语义 (Algorithm 13 片段 + Algorithm 8)

Host 在生成 `src` [8,256] 时, 须等价于下列过程 (d 为 32 字节 `derand`, `SEED_D` 文档化, 默认建议 20260619):

1. $(\rho, \sigma) \leftarrow \mathbf{G}(d \parallel k)$ (Alg.13 行 5; k 按标准编码为单字节)。
2. $N \leftarrow 0$ 。
3. **对** $i = 0, \dots, k-1$: $s_i \leftarrow \text{SamplePolyCBD}_{\eta_1}(\text{PRF}_{\eta_1}(\sigma, N))$; $N \leftarrow N+1$ 。(Alg.13 行 7)
4. **对** $i = 0, \dots, k-1$: $e_i \leftarrow \text{SamplePolyCBD}_{\eta_2}(\text{PRF}_{\eta_2}(\sigma, N))$; $N \leftarrow N+1$ 。(Alg.13 行 8)

SamplePolyCBD₂ (Alg.8): 由均匀随机字节生成系数, 每个系数为两个比特之和减去两个比特之和 (中心二项, 支撑于 $\{-2, \dots, 2\}$), 表示为 Z_q 非负规范整数 (与 `liboqs/mlkem-native/mlk_poly_cbd2` 一致)。

3.3 `src.bin` 布局 (与 2s1e 设备契约一致)

行	内容	要求
0..3	$s_0, \dots, s_3$	四条 独立 CBD 采样; 禁止 <code>np.tile</code> 同一行
4..7	$e_0, \dots, e_3$	四条 独立 CBD 采样; 禁止 $(e+p) \bmod q$ 人工变体

设备 Stage1 仍将 `s` 复制到双 AIV 行块; NTT 后 $\hat{s}_0, \dots, \hat{s}_3$ **一般互不相同** (不再要求 $\hat{s}_0 = \dots = \hat{s}_3$)。

3.4 参考实现与交叉验证 (`gen_data` 登记)

来源	用途
thirdparty/liboqs/ ML-KEM-1024	黑盒 oracle: 相同 d 下 s_i, e_i 系数对照 (推荐)
thirdparty/liboqs/.../ sampling.c	mlk_poly_cbd2 语义
ascendc-tests/.../vec- k4-v2/scripts/gen_data .py	仅借鉴 NTT golden 链; 禁止 build_src_se() 采样逻辑

gen_data 须打印: SEED_D、8 行唯一性检查 (assert 8 行两两不等, 或记录 max_abs_diff 供审计)。

4 FIPS 203 Algorithm 13 行 16–20 设备语义

4.1 行 16–17 (NTT)

对 $i=0, \dots, k-1$: $\hat{s}_i \leftarrow \text{NTT}(s_i)$, $\hat{e}_i \leftarrow \text{NTT}(e_i)$ 。本仓以 Tag5T 三段式实现 (非内核蝶形 MlkemNtt): Stage1 limb6 $\rightarrow$ Stage2 int8 MMAD $\times 4 \rightarrow$ Stage3 平面 merge + stage31_mod。

4.2 行 18 (NTT 域内积 + 噪声)

对每个 $p \in \{0, \dots, k-1\}$:

$$\hat{t}[p] = \text{mod}_q \left(\sum_{j=0}^{k-1} \hat{A}[p, j] \circ \hat{s}[j] + \hat{e}[p] \right)$$

其中 $\circ$ 为 NTT 域多项式乘 (Alg.11 MultiplyNTTs); j 循环内 **不** final mod; $\hat{e}$ 以向量 Add 累加后一次 mod_q_final_vec (与 v2 stageHatDotOnly 一致)。

4.3 行 19–20 (ByteEncode₁₂)

数学 (FIPS 203 Alg.5): 对每个系数取低 12 bit, 按标准交织为每 poly **384 B**; $k=4$ 时 $\text{ek} \leftarrow \text{ByteEncode}_{12}(\hat{t})$ 、 $\text{sk} \leftarrow \text{ByteEncode}_{12}(\hat{s})$, 各 **1536 B** (4×384)。

设备入口: Aiv2s1eUbPipeline::stageEncodeOut (与 v2 相同); 对 $p \in \{0, \dots, k-1\}$:

1. $\hat{t}[p]$ (UB ub_that) $\rightarrow$ poly_byte_encode12_local $\rightarrow$ GM ek 切片;
2. $\hat{s}[p]$ (UB ub_ntt, 行索引 s_row(p)=p) $\rightarrow$ 同上 $\rightarrow$ GM sk 切片。

双 AIV 分片 (与行 18 一致, **禁止**单 AIV 写满 polyvec):

- AIV0: $p \in \{0, 1\}$, 各编 1 poly $\hat{t}$ + 1 poly $\hat{s} \rightarrow$ 本地 768 B + GM 偏移 $p \cdot 384$;
- AIV1: $p \in \{2, 3\}$, 同上。

宏	路径
VEC=1, PREFETCH=1 (默认)	poly_byte_encode12_prefetch_local: GM ROM Gather 索引 → UB; 128-wide Gather×2 解交错; compute_b012; 384 B DataCopy
VEC=1, PREFETCH=0	poly_byte_encode12_vec_local: tile=32, 每 tile CreateVecIndex+Gather (legacy, 仅对照)
VEC=0	标量 poly_byte_encode12_scalar

每 AIV UB 仍握完整 $\hat{s}[0..3]$ (Stage1 复制), 但 sk 只写本核 p 区间。

实现路径 (2026-06-19 与 v2 对齐): 默认 BYTE_ENCODE12_PREFETCH=1 (整 poly prefetch), 非旧版 tile=32 循环。分发 poly_byte_encode12_local:

关键文件(fork 自 v2 / byteencode12-vec-k4 探针): byte_encode12_config.hpp, byte_encode12_pair.h, byte_encode12_vec.hpp, byte_encode12_rom_tables.*, byte_encode12_ub_load.hpp; 内核 mmad_custom.cpp 在 AIV 编译单元 #include byte_encode12_rom_tables.cpp (与 Alg.11 ROM 相同模式, 避免 MIX 双份符号)。

UB scratch: HAT_LINE18_FULLPOLY=1 时 encode 工作区在 dot scratch 末尾; PREFETCH=1 须 kPrefetchScratchBytes=5504 (高于 tile 路径 1664 B)。stageEncodeOut 循环体与 v2 一致, 仅 scratch 预算与 encode_ws 槽位绑定变更。

Golden: Host byte_encode12_ref; I/O 与 v2 mixPass=7 preset 字节级一致 (见 encode-only 探针 byteencode12-vec-k4)。

性能参考 (SIM, 910B4, 非门禁): v2 全链路 prefetch ~77958 tick (encode 边际 ~+12421 vs 无 encode); encode-only 探针 prefetch ~17429 tick。本 exp fork 同步 v2 后 tick 应同量级 (FIPS CBD 输入不改变设备 encode 路径)。

5 计算语义：三段式 NTT + 平面 mat_c

5.1 流水线

1. **Stage1** (AIV Aiv2s1eSplit): $src [8, 256] \rightarrow S0 \text{ limb6 int8}$; 双 $\hat{s}$ 行块 + $\hat{e}$ 对半。
2. **Stage2** (AIC AicMmad×4): $even/odd \times lo/hi \rightarrow$ 四路临时 int32。
3. **Stage2 后 pack** (AIV Aiv2s1ePackMatCPlanar): $\rightarrow mat_c_planar [96, 128]$ 。
4. **Stage3 + 行 18 + 行 19–20** (AIV Aiv2s1eUbPipeline): 平面 merge $\rightarrow \hat{s}/\hat{e}$ UB; $j \rightarrow p$ 内积 $\rightarrow \hat{t}$ UB; ByteEncode $\rightarrow GM \text{ ek/sk}$ 。

5.2 平面行号 (禁止混用)

$$planar_row(slot, limb, half) = half \cdot 48 + slot \cdot 4 + limb$$

slot: 0..3 $\hat{s}$ AIV0; 4..7 $\hat{s}$ AIV1 (同值复制); 8..9 $\hat{e}$ AIV0; 10..11 $\hat{e}$ AIV1。

NTT S1–S3 禁止 Gather; 仅 bulk DataCopy 四 limb 行。

6 数据搬运与 UB 不变量

6.1 驻留 vs 落盘

- **驻留**: S3 后 $\hat{s}$ 、 $\hat{e}$ 、 $\hat{t}$ 在 UB (`ubNttBuf_`/`ubThatBuf_`) 直至 ByteEncode 结束 (§4.3)。
- **ByteEncode 输入**: $\hat{t}[p]$ 、 $\hat{s}[p]$ 已在 UB; **禁止** encode 阶段再从 GM 读 `dst/t_hat` preset (全链路 `mixPass=0`)。
- **落盘**: 对拍 `dst/t_hat` dump 仅在阶段结束; `ek/sk` 为算法 GM 输出。
- **禁止**: S3 后 $\hat{s}$ GM 绕路; **禁止** SHAT_PEER; **禁止** 嵌套第二 TPipe。

6.2 行 18 数据面

- $\hat{A}[p, j]$: 只读 GM `a_hat` (行主序 $(p \cdot K + j) \cdot N + c$)。
- $\hat{s}[j]$: 只读 UB `ub_ntt` (每 AIV 握完整 $j \in [0, 3]$)。
- Alg.11 ROM (γ /Gather/interleave): Init DataCopy 进 UB, Compute 禁止 SetValue 重填。

7 MIX 调度与 mixPass

7.1 FSM (blockDim=1)

1. AIV_SPLIT: Stage1; 末 SET。
2. AIC_MMAD: WAIT; MMAD $\times 4$; 末 SET。
3. AIV_PACK: WAIT; 平面 pack。
4. Aiv2s1eUbPipeline: S3 / 行 18 / Encode (无额外核间同步)。

7.2 mixPass 调试

与 v2 IMPLEMENTATION_REFERENCE.md §4 相同: 0= 全链路; 1..7 分阶段; CPU 可 5→4 check-point。

8 AI Core 规模 (强制专节)

项	本用例约定
核类型	KERNEL_TYPE_MIX_AIC_1_2 (1 Cube + 2 Vector / 1 AI Core)
计划 AI Core 数	仅 1 : <code>blockDim=1</code> ; <code>run.sh --aicore 1</code>
数据划分	AIV0: $p \in \{0, 1\}$, $\hat{e}$ 行 4..5; AIV1: $p \in \{2, 3\}$, $\hat{e}$ 行 6..7; 双核各握完整 $\hat{s}[0..3]$
选型理由	与 v2 探针已验证路径一致; 单趟 launch 完成行 16–20; 多核须另开 spec

多档验证 首版不测；晋级 stable 前可增 aicore 档位

9 I/O 与 Golden 契约

9.1 文件 (gen_data 与 main 逐字一致)

路径	形状	dtype	语义
input/src.bin	[8, 256]	int32	§3 FIPS CBD s e
input/a_hat.bin	[16, 256]	int32	$\hat{A}$; SEED_AHAT=20260615
input/lut_even/odd_stage128.in	[512, 128]	int8	Tag5T LUT (transpose_mlkem_luts_i8.h)
input/tiling.bin	(256, 4, <i>mixPass</i>)	int32	与 v2 tiling.h
output/dst.bin	[12, 256]	int32	NTT 后 $\hat{s}/\hat{e}$
output/t_hat.bin	[4, 256]	int32	行 18
output/ek_polyvec.bin	4×384	uint8	行 19
output/sk_polyvec.bin	4×384	uint8	行 20
output/golden_*.bin	同上	同上	Host 参考

9.2 Golden 生成链 (NTT 与 v2 同构)

```
src (FIPS CBD s||e)
-> encode_2s1e_s0 -> mat_c_tmp_golden -> pack_mat_c_planar
-> golden_dst_from_planar (merge + stage31_mod)
-> hat_inner_product_ref (dot / add)
-> byte_encode12_ref (ek/sk)
```

禁止在 gen_data 内对 NTT 逐行调用 C MlkemNtt(); 须与设备三段式同构 (与 v2 §2.1 相同)。

9.3 Golden 与设备 mod 解耦

行 18 golden: HAT_GOLDEN_MOD_VARIANT=0 (int64 floor); 设备: F203_MOD_VARIANT=1 Barrett 向量。对拍比最终 int32 多项式。

9.4 验收

```
bash run.sh -r cpu -v Ascend910B4
bash run.sh -r sim -v Ascend910B4
python3 scripts/verify_result.py
```

默认即全链路 (HAT_LINE18_DOT_ONLY=0、HAT_BYTE_ENCODE=1、BYTE_ENCODE12_PREFETCH=1, 见 run.sh); 调试分段须显式覆盖。要求 max_abs_diff=0。SIM 须 source scripts/camodel_sim_log.sh。

10 AscendC 实现规划

10.1 参照 fork 顺序

1. 壳: 自 `ascendc-tests/pass-fix-f203-2s1e-alg13-16171820-vec-k4-v2/` 复制 `mmad_custom.cpp`、`2s1e_post_ntt_ub.hpp`、`aiv_func.hpp`、`tiling.h`、`alg11_*`、`hat_*`、`byte_encode12_*` (含 `rom_tables.*`、`ub_load.hpp`)、`cmake/`、`run.sh`。
2. **ByteEncode 宏**: 默认 `BYTE_ENCODE12_VEC=1`、`BYTE_ENCODE12_SCATTER_VEC=1`、`BYTE_ENCODE12_PREFETCH=1` (与 `v2 run.sh` 一致)。
3. 改 `scripts/gen_data.py`: 实现 §3; 删除 `FIXED_POLY/NTTS2S1E_E_POLY_SEED` 探针逻辑。
4. 改 `verify_result.py`: 保留全链路对拍; 移除「`$` 四行相同」探针专用断言 (或改为可选 `--probe-layout`)。
5. 内核文件名建议: `f203_alg13_16171820_2s1e_custom.cpp` (与探针 `mmad_custom` 区分)。

10.2 计划目录 (实现阶段)

```
exp-fips203-mlkem-pke-alg13-16171820-2s1e-k4/
*-实现方案-customspec.tex / .pdf
STATUS.md
f203_alg13_16171820_2s1e_custom.cpp
main.cpp, CMakeLists.txt, run.sh
scripts/gen_data.py, verify_result.py
(headers forked from vec-k4-v2)
```

10.3 分步验收

步	内容	验收
S0	本 customspec PDF	文档就绪
S1	<code>gen_data</code> : FIPS CBD s/e + golden 链	liboqs 系数对照; 8 行互异
S2	fork v2 内核 + 新 src	CPU <code>max_diff=0</code>
S3	SIM 全链路	tick 参考 $< 10^5$; v2 prefetch 基准 ~ 77958 (附录, 非门禁)
S4	npu	有实机时

11 全量 API 清单 (主路径, CANN 9.0.0)

说明: 分类按 PDF 第 2 章; 在线页为 `atlasascendc_api_07_*`。

API	分类	在线 ID	A2	本用例 dtype	备注
KERNEL_TASK _TYPE_DEFAULT LT	Utils	编程指南	✓	MIX_AIC_1_2	融合核
CrossCoreSe tFlag	同步	07 列表	✓	event id	AIV→AIC
CrossCoreWa itFlag	同步	07 列表	✓	event id	AIC←AIV
PipeBarrier	同步	2.3	✓	PIPE_ALL/MTE2	KYBER_PIPE_ALL
DataCopy	2.3.1	07_0101	✓	int32/int8 GM↔UB	平面四 limb; 无 Gather
ShiftRight	2.3.3	07_0059	✓	int32	Stage1 <i>hi</i>
Muls	2.3.3	07_0055	✓	int32	Stage1 <i>hi</i> ·64
Sub	2.3.3	07_0036	✓	int32	Stage1 <i>lo</i>
Add	2.3.3	07_0035	✓	int32	行 18 lazy Σ ; $+\hat{e}$
Cast	2.3.3	07_0073	✓	i32→i16→half→i8	Stage1 窄化链
Gather	2.3.3	07 列表	✓	int32 索引	Alg.11 basemul; ByteEncode 解交 错 (§4.3); NTT S1–S3 禁用
TPipe/TQue ue/TBuf	2.3	07_0108	✓	UB 资源	单 slim TPipe
AicMmad	Cube	merged_kyber	✓	int8×int8→int32	Stage2 ×4

11.1 评估过但未采用

API/路线	结论
Matmul<> / REGIST_MATMUL_0B	NTT 融合已 确定
SHAT_PEER GM 交换	2s1e 每核握 完整 $\hat{s}$
竖堆 mat_c + Gather S3	平面 bulk 取 代
探针 FIXED_POLY 输 入	本 exp 禁止

12 逐步覆盖率评估

段	数学/操作	API	覆盖
S1	limb6 拆分	ShiftRight/Muls/Sub/Cat	100% 向量
S2	int8 MMAD	AicMmad	Cube
S3	merge+mod	DataCopy+RouteA 向	100% (无
		量	Gather)
18	Alg. $11\times k + \Sigma + \hat{e} +$ mod	compute_on_ub/Add/mod160%in	100% 向量 vec
19-20	ByteEncode ₁₂	poly_byte_encode12_prefetch100%in	100% 向量 local
		(默认)	ROM+128
			Gather; 双
			AIV 各 2 poly
Host CBD 采样			Host Python
			(非 AI Core)

标量缺口：gen_data 中 FIPS G/PRF/CBD；CPU_DEBUG 下行 18 标量回退（与 v2 相同）。

13 附录：SIM tick 参考 (910B4, 非门禁)

配置 (v2 / 同步后 exp)	Total tick
全链路 + BYTE_ENCODE12_PREFETCH=1 (默 认)	~77958
全链路 + PREFETCH=0 (tile32 legacy)	~85991
encode-only 探针 prefetch	~17429

FIPS CBD 输入仅改变 Host src；设备 encode 路径与探针 v2 相同。本 exp 在 2026-06-19 前 fork 的 tick ~86111 为 tile32 时代测量；同步 prefetch 后应降至 ~77958 量级。

14 已废弃路径

- ascendc-tests/frozen/frozen-fix-f203-2s1e-alg13-16171820-vec-k4/ half-basemul 路线。
- v2 build_src_se() 的 FIXED_POLY 与 (e+p) 变体——本 exp 明确禁止。

15 关联文档

- 探针：ascendc-tests/pass-fix-f203-2s1e-alg13-16171820-vec-k4-v2/IMPLEMENTATION_REFERENCE.md

- 原理: docs/notes/F203-2s1e-NTT内积UB融合技术总结.md
- ByteEncode: docs/notes/F203-ByteEncode12-prefetch技术总结.md; qa/2026-06/2026-06-19-ByteEncode12-only探针与prefetch实验.md
- 讨论: qa/2026-06/2026-06-18-内积布局与NTT内积UB融合讨论.md
- 标准: library/documents/NIST.FIPS.203.pdf
- liboqs 对照: thirdparty/liboqs/